NFPL Access Control Service — Design Note

1. Goal and assumptions

The service makes an explicit decision about a user, an action and a specific shipment (job). The business rule is not simply “is the user an admin?” A normal decision must account for the user’s current status, assigned role, client, branch, direction, transport mode, workflow stage, and separation of duties.

Assumptions for this prototype:

- Permission assignments are additive. A permission row grants a role within the intersection of its non-empty scope dimensions; an empty dimension means “all” for that dimension.
- Confirmed → Prepared → Checked → Filed is the normal workflow. Reopened jobs return to Maker preparation. Cancelled and Filed jobs require special handling to be reopened or, in the Filed case, cancelled.
- A user marked absent cannot act or receive an actionable queue. The absence itself does not silently grant the absent user’s powers to someone else; a qualified substitute receives a temporary, reasoned grant with an expiry.
- Client users are read-only and may only access their own client. The automated parser is a service identity, not an employee.
- Unknown actions, inactive users, missing permissions, unmatched scopes and unsupported state transitions are denied by default.

2. Model and enforcement

The database has separate tables for users, clients, branches, jobs, role_definitions, role_action_permissions, permission_assignments and audit_logs.

A permission_assignments row combines a user, role, optional client/branch/direction/mode restrictions, an effective-from timestamp, an optional expiry, a revocation timestamp and a reason. Rows are revoked rather than deleted, so historical permissions remain inspectable. Each non-empty scope field must match the job. Multiple grants are allowed, so one employee can be Maker everywhere and Checker only for selected clients.

Role definitions and role-to-action mappings live in PostgreSQL. An access administrator can create a custom role from an approved, existing action catalogue without a code deployment. Code still owns the workflow invariants (for example, a Checker can act only when a job is Prepared, and the preparer cannot check their own work). New *actions* or sensitive exception capabilities require code review; custom roles cannot gain permission-management or filed-job override actions.

The policy lives in one pure function, canPerform(user, action, job, assignments, context, time). It returns allowed, a stable decision code and a human-readable explanation. HTTP routes call this policy; they do not trust the caller’s claim that the action is allowed. Workflow changes lock the job row, re-check authorization immediately before updating, and save the state change and audit entry in one database transaction.

3. Decisions for the 14 assessment questions

1. Visibility vs action

Visibility is itself an explicit view_job action. Internal users can see a job only if an effective scoped role grants view_job. Client users are limited to their client and read-only access. The global access administrator manages permissions but does not automatically see or alter every shipment. A user may be able to view a job but not prepare, check or file it.

2. Different roles for different parts of the business

Use multiple scoped assignments, not a single global role field. Priya has a Maker assignment with all scope dimensions blank and a separate Checker assignment restricted to BluePeak Retail. Rahul has Checker for Aster Imports at Mumbai, direction IMPORT, any mode. The dimensions in a single row are combined with AND; separate rows are independent grants.

3. Maker is also qualified to Check

The system records prepared_by_user_id and refuses check_job when it matches the acting user ID, regardless of whether the person also has a Checker grant. Qualification does not override separation of duties. Another scoped Checker must check the work.

4. Responsible person is absent

An absent user is denied actions and removed from the actionable queue. A supervisor chooses an already qualified substitute and grants the smallest required scope with an expiry and reason. This is auditable and avoids copying all of the absent employee’s powers. The sample includes an absent Checker and a temporary Checker grant that expires seven days after seeding.

5. Responsibilities change while a job is in progress

Permission assignments carry effective_from, expires_at and revoked_at; each request reads current database permissions rather than relying on an application cache. A grant change applies to the next authorization check. Being halfway through a job does not preserve a revoked permission. Existing assignments remain in history so an investigation can evaluate which grants were effective at a chosen time.

6. Cancel a Filed job or reopen a closed job

Routine cancellation is limited to scoped operational roles and only before filing. A filed job can be cancelled only through workflow_admin; reopening a Filed or Cancelled job also uses workflow_admin. Both actions require a meaningful reason and produce an audit event with before/after state, actor, time and request ID. Access administration and workflow overrides are separated. This MVP does not implement a second approver; a production deployment should add dual approval for high-impact reopening.

7. Client employee follows shipments

A client employee is a client user with a client_viewer grant scoped to their client. The service denies jobs for other clients and does not expose mutation actions to the client user. The API still requires verified authentication in a production system.

8. Admin fixes a stuck job; admin self-escalation

There are separate roles for access_admin and workflow_admin. An access administrator manages permission grants but has no implicit shipment-write bypass. A workflow administrator may use high-risk exception actions with a reason and audit event. Administrators cannot create, revoke or broaden their own grants; a second administrator must do it. Custom roles cannot assign manage_permissions, cancel_filed_job or reopen_filed_job.

9. Overnight automated parser

The parser is a dedicated service identity with a scoped parser role. It can create an initial Confirmed job only inside its grant scope, or update a narrow metadata whitelist (externalReference, description, etaDate) on non-filed, non-cancelled jobs. It cannot change workflow stage, who prepared/checked/filed, or alter a finalized job. Parser operations are audited like human mutations.

10. “Why can she do this and he can’t?”

GET /authorization/check evaluates a selected user/action/job and returns allowed, a stable reason code and a plain-English explanation. A global access administrator can inspect another user’s decision; other users can inspect their own. The API uses the same policy used to authorize changes, not a separate explanation-only ruleset.

11. “Who could have changed this job on the 3rd?”

Job history stores actor, action, timestamp, before/after snapshots, reason and request ID. Permission grants are retained with effective and revocation times. The investigation endpoint reconstructs the job snapshot at the requested time from later audit events and evaluates each recorded assignment as of that timestamp. This answers who was eligible under the stored policy; the job history answers who actually changed it. **Known limit:** user active/absent status changes are not currently versioned, so the historical check conservatively uses current status and identifies this limitation in its response. The demo should show both event history and eligible-user report.

12. Introduce a new kind of role while engineering is busy

An access administrator can call POST /roles with a new role name, description, user type and a list of approved actions. Role definitions and action mappings are stored in PostgreSQL, so adding a new combination of existing low-risk actions requires no code deployment. Scope is then granted with POST /permissions. High-risk actions and permission administration are deliberately excluded from custom roles; changing the action catalogue or the workflow invariants remains an engineering-controlled change.

13. “Jobs waiting for me” count matches what the person can actually do

The queue query uses the current permission assignments, role-action map, scope fields, user status and relevant workflow stage. It excludes a Checker from jobs they prepared themselves. Filtering and pagination happen in PostgreSQL rather than loading 200,000 jobs into Node. Indexes support stage/scope lookup, user-role lookup and recent audit retrieval. Permission mappings are queried live rather than cached so changes take effect promptly.

14. What if the system is unsure?

Fail closed. Missing user/job, no matching effective assignment, unknown action, invalid role scope, expired/revoked grant, inactive user, self-check conflict or invalid stage results in denial with a reason code. Database failures prevent the state change; workflow mutation and audit write share a transaction so one cannot commit without the other.

4. Alternatives and trade-offs

Node.js is chosen because it matches the developer’s JavaScript experience and supports asynchronous request/database I/O. Express supplies straightforward routing and middleware without requiring a more opinionated framework. PostgreSQL models relational entities and supports foreign keys, transactions and indexes. The pg library keeps SQL explicit and parameterized. Other reasonable stacks include Python/FastAPI or Java/Spring Boot and MySQL/MongoDB; the choice is about fit and implementation cost, not claiming those alternatives are invalid.

This prototype optimizes for explainability and assessment scope rather than a full enterprise identity system. It does not include a frontend, SSO, dual-approval workflow, historical user-status table or a full integration/performance test run. The demo actor header exists only to make sample actions reproducible. Production must use a verified identity provider and should add dual approval, security-event monitoring, retention policy, migration tooling and database-backed integration/load testing.